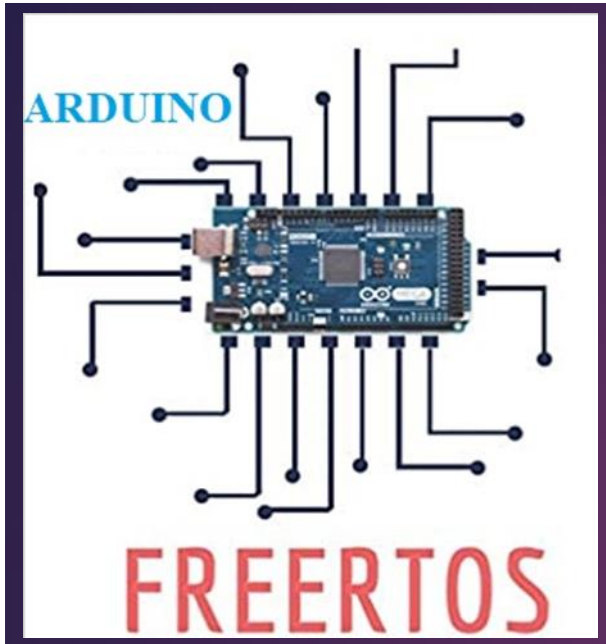


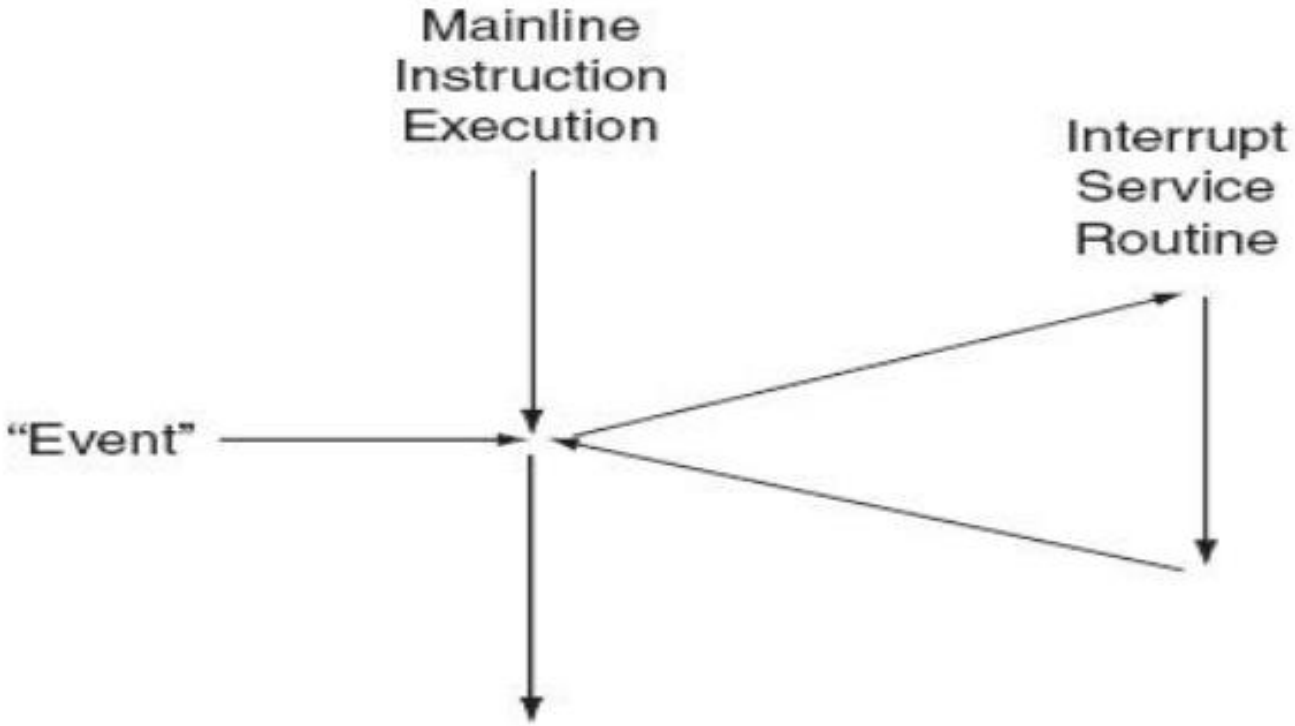
Chương 3:
Quản lý ngắt trong FreeRTOS

Interrupt Management

- Giới thiệu về ngắt
- Quản lý ngắt dùng Queue
- Quản lý ngắt dùng Semaphore

Giới thiệu về ngắt: Khái niệm

- Ngắt là sự kiện dừng công việc hiện tại của CPU, buộc CPU thực hiện một việc nào đó rồi mới quay trở lại thực hiện tiếp công việc cũ.
- Trong hệ thống nhúng, khi có tín hiệu Interrupt, tác vụ đang chạy sẽ bị ngừng lại, thay vào đó Interrupt Service Routine (ISR) sẽ được thực thi. Sau khi ISR hoàn thành công việc, tác vụ ban đầu sẽ được thực thi tiếp

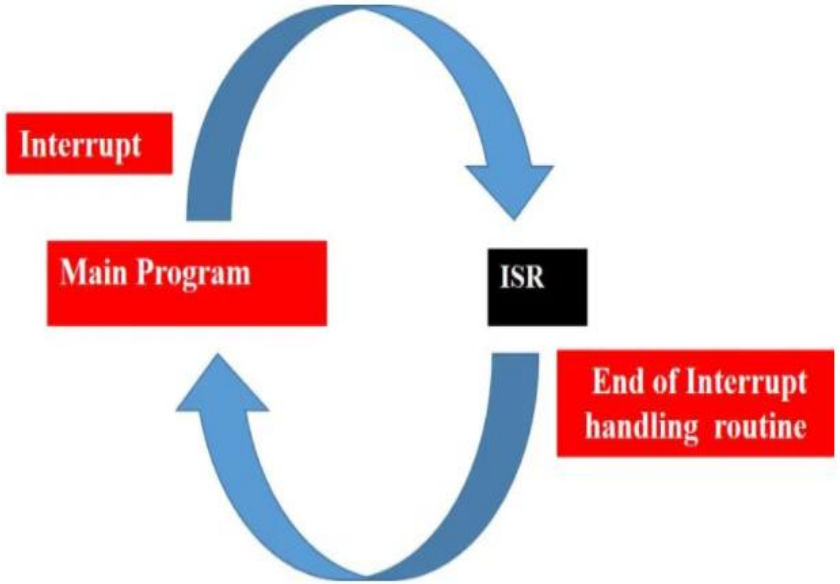


Giới thiệu về ngắt: Khái niệm

- Các hệ thống nhúng thời gian thực thực hiện các hành động để phản hồi sự kiện khởi nguồn từ môi trường.
  - Ví dụ, một gói tin đến ngoại vi Ethernet (sự kiện) yêu cầu phải được chuyển qua ngăn xếp TCP/IP để xử lý (hành động).
- Các hệ thống lớn (non-trivial system) phải phục vụ các sự kiện khởi nguồn từ nhiều nguồn:
  - Tất cả đều sẽ có các yêu cầu về thời gian phản hồi và xử lý khác nhau.
  - Mỗi trường hợp, phải được thực hiện theo cách xử lý sự kiện tốt nhất

Giới thiệu về ngắt: Khái niệm

- ISR (Chương trình phục vụ ngắt) còn được gọi là trình xử lý ngắt.
  - ❑ Có nhiều loại trình xử lý ngắt khác nhau có thể xử lý nhiều loại ngắt khác nhau
- ISR sẽ sử dụng loại biến (volatile variable) có thể được sử dụng giữa các đoạn mã khác, ISR phải ngắn và nhanh nhất có thể. Ngắt thực thi ngay lập tức việc dừng chương trình hiện đang thực thi để nhảy vào hàm ngắt và thực thi mã của ISR



Giới thiệu về ngắt: Khái niệm

- FreeRTOS không áp đặt bất kỳ chiến lược xử lý sự kiện cụ thể nào trên trình thiết kế ứng dụng, nhưng cung cấp các function cho phép chiến lược được chọn được thực hiện một cách đơn giản và có khả năng bảo trì
- Chú ý chỉ các hàm và macro API kết thúc bằng “FromISR” hoặc “FROM_ISR” mới được sử dụng trong chương trình phục vụ ngắt (***ISR – Interrupt Service Routine***).

Giới thiệu về ngắt: Phân biệt với task

- Trong FreeRTOS, cần phân biệt sự khác nhau giữa Task và Interrupt:
  - **Task**: là một "software feature" của FreeRTOS, không có sự khác biệt giữa các dòng MCU khác nhau mà FreeRTOS hỗ trợ. Task được quản lý bởi Scheduler (cũng là một thuật toán). Các thiết lập dành cho Task xảy ra ở tầng OS (cụ thể là FreeRTOS) bởi các API mà OS cung cấp;
  - **Interrupt Service Routine**: là một "hardware feature" của MCU. Các thiết lập dành cho Interrupt xảy ra ở tầng hardware bằng cách thay đổi các giá trị trong thanh ghi (register) của MCU
- Interrupt luôn có độ ưu tiên cao hơn Task và không có cách nào để Task có thể chiếm quyền thực thi của ISR
- FreeRTOS có 2 hằng số quan trọng quy định độ ưu tiên của interrupt:
  - **configMAX_SYSCAL_INTERRUPT_PRIORITY** – độ ưu tiên tối đa của interrupt mà các interrupt-safe API có thể được sử dụng từ trong ISR;
  - **configKERNEL_INTERRUPT_PRIORITY** – độ ưu tiên của RTOS kernel, luôn phải là độ ưu tiên thấp nhất trong các interrupt, giá trị này phụ thuộc vào từng dòng MCU

Giới thiệu về ngắt: Mức ưu tiên

- **Critical Section (CS hay Critical Region)** là vùng mà tất cả interrupt có mức độ ưu tiên thấp hơn hoặc bằng với **configMAX_SYSCAL_INTERRUPT_PRIORITY** sẽ bị vô hiệu hóa.
  - **CS** được bao bọc bởi lệnh **taskENTER_CRITICAL**() và **taskEXIT_CRITICAL**().
  - Ở bên trong **CS**, Scheduler sẽ ngừng hoạt động, do đó Context Switching sẽ không xảy ra.
  - Đối với các interrupt bị vô hiệu hóa bên trong **CS**, ISR của nó sẽ được thực thi ngay khi kết thúc **CS**.
- Những interrupt có độ ưu tiên cao hơn **configMAX_SYSCAL_INTERRUPT_PRIORITY** không bị ảnh hưởng ở bên trong các CS, nhưng không thể sử dụng các interrupt-safe API từ ISR của nó

Giới thiệu về ngắt: Arduino Interrupts

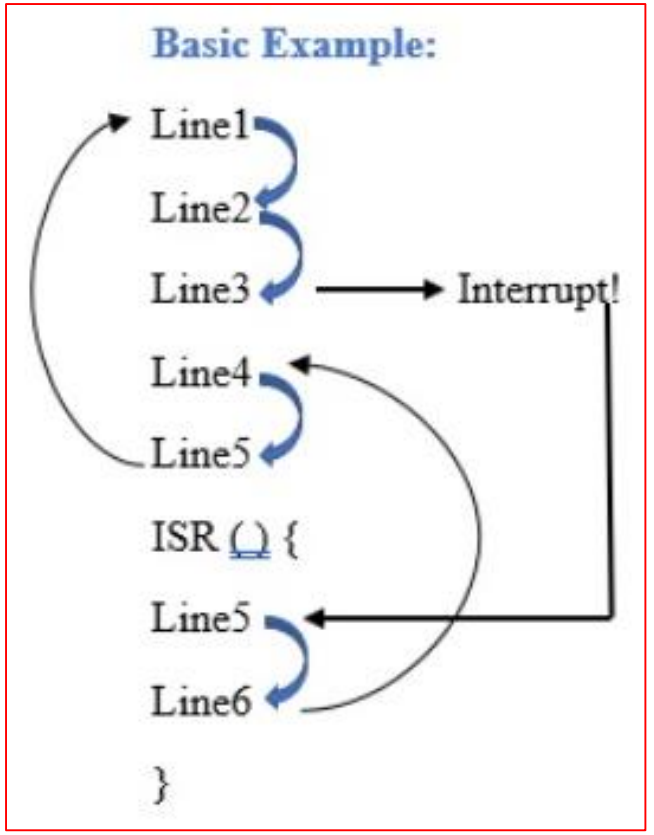
BOARD	DIGITAL PINS USABLE FOR INTERRUPTS
Uno, Nano, Mini, other 328-based	2, 3
Uno WiFi Rev.2, Nano Every	all digital pins
Mega, Mega2560, MegaADK	2, 3, 18, 19, 20, 21
Micro, Leonardo, other 32u4-based	0, 1, 2, 3, 7
Zero	all digital pins, except 4
MKR Family boards	0, 1, 4, 5, 6, 7, 8, 9, A1, A2
Nano 33 IoT	2, 3, 9, 10, 11, 13, 15, A5, A7
Nano 33 BLE, Nano 33 BLE Sense	all pins
Due	all digital pins
101	all digital pins (Only pins 2, 5, 7, 8, 10, 11, 12, 13 work with **CHANGE**)

- https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/

Giới thiệu về ngắt: Arduino Interrupts

Tham chiếu hình:

- Từng dòng mã được thực thi cho đến khi ngắt được gọi trên Line 3:
- Sau đó nhảy xuống ISR và bắt đầu thực hiện Line 5 và Line 6
- Sau khi thực hiện các dòng trong ISR nó nhảy trở lại Line 4 và hoàn thành việc thực hiện như thường lệ.
- Nếu chương trình ở trong vòng lặp, thì tiếp tục quay lại Line 1.



Giới thiệu về ngắt: Arduino Interrupts

Các ngắt được kích hoạt như thế nào?

- Trong các ngắt Arduino, có thể đặt cách kích hoạt (trigger) các ngắt.
- Có năm loại kích hoạt ngắt Arduino:
  - CHANGE: Khi tín hiệu thay đổi ngay cả khi tín hiệu tăng hoặc tín hiệu giảm hoặc nếu tín hiệu ở trạng thái thấp ở 0 hoặc nếu tín hiệu ở trạng thái cao thì kích hoạt 5v.
  - RISING: Trên một cạnh tăng, tín hiệu đi từ thấp đến cao có nghĩa là tín hiệu kích hoạt từ 0v đến 5v.
  - FALLING: Trên một cạnh giảm, tín hiệu đi từ cao xuống thấp có nghĩa là tín hiệu được kích hoạt từ 5v đến 0v.
  - LOW: Thấp là sự kích hoạt liên tục bất cứ khi nào tín hiệu ở mức thấp hay nói cách khác là tín hiệu trên 0v.
  - HIGH: High là sự kích hoạt liên tục bất cứ khi nào tín hiệu ở mức cao hay nói cách khác là tín hiệu trên 5v.
- 1 trong số 5 loại này sẽ được đính kèm ngắt và chỉ định mã pin:
  - Ví dụ: chân số 2 ứng với ISR là function sẽ được gọi và chế độ cho biết rằng khi nào ngắt được kích hoạt.

Giới thiệu về ngắt: Arduino Interrupts

Làm thế nào để sử dụng ngắt Arduino?

- Cách sử dụng các hàm chức năng ngắt Arduino đã có sẵn trong Arduino IDE để khởi tạo ngắt Arduino:

attachInterrupt(digitalPinToInterrupt(pin), ISR, mode) ;

- Hàm này nhận 3 đối số:
  - Ngắt tại chân tương ứng với số pin mà ta muốn sử dụng để kích hoạt ngắt bên ngoài.
  - Tên hàm ISR mà ta muốn gọi mỗi khi có ngắt.
  - Kiểu ngắt đang sử dụng (Change, Rising, Falling, Low, High)
- Cần xác định muốn khởi tạo nó bằng chân nào của Arduino, đính kèm hàm chức năng ngắt ISR được sử dụng
  - Ví dụ: attachInterrupt(digitalPinToInterrupt(2), InterruptFunction, Low) ;

Giới thiệu về ngắt: Arduino Interrupts

- Ví dụ:

```
const byte LED = 13;
const byte SWITCH = 2;

// Interrupt Service Routine (ISR)
void switchPressed ()
{
  if (digitalRead (SWITCH) == HIGH)
    digitalWrite (LED, HIGH);
  else
    digitalWrite (LED, LOW);
}  // end of switchPressed

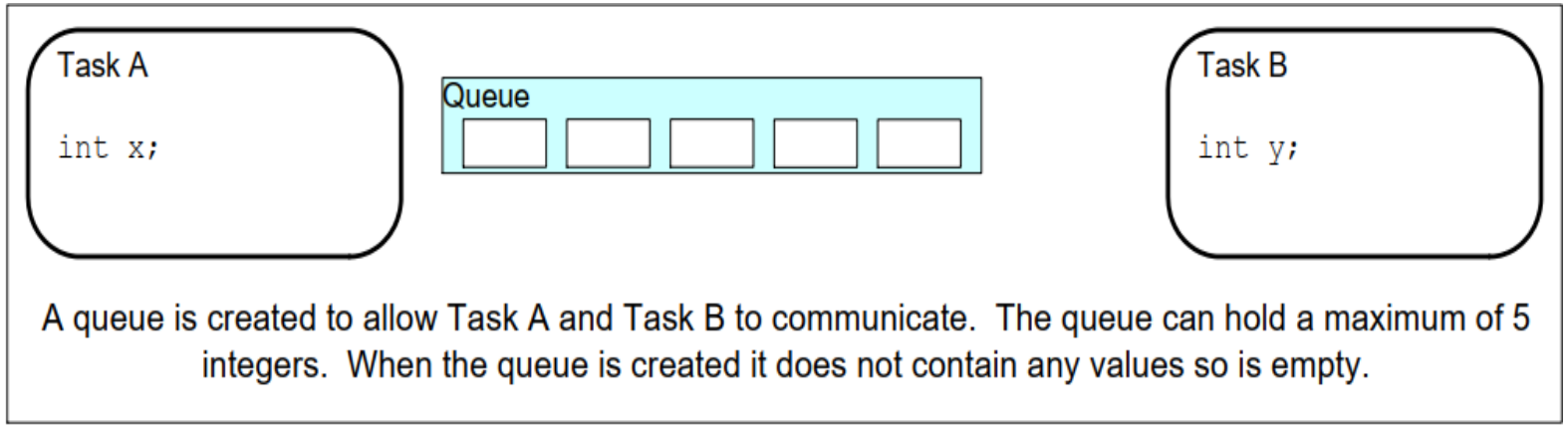
void setup ()
{
  pinMode (LED, OUTPUT);  // so we can update the LED
  pinMode (SWITCH, INPUT_PULLUP);
  attachInterrupt (digitalPinToInterrupt (SWITCH), switchPressed, CHANGE);
}  // end of setup

void loop ()
{
  // loop doing nothing
}
```

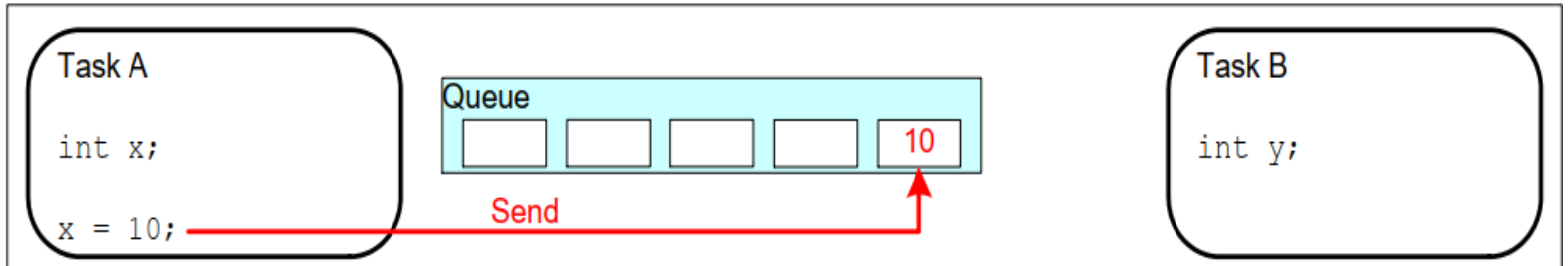
Interrupt Management

- Giới thiệu về ngắt
- **Quản lý ngắt dùng Queue**
- Quản lý ngắt dùng Semaphore

Queue và xử lý ngắt

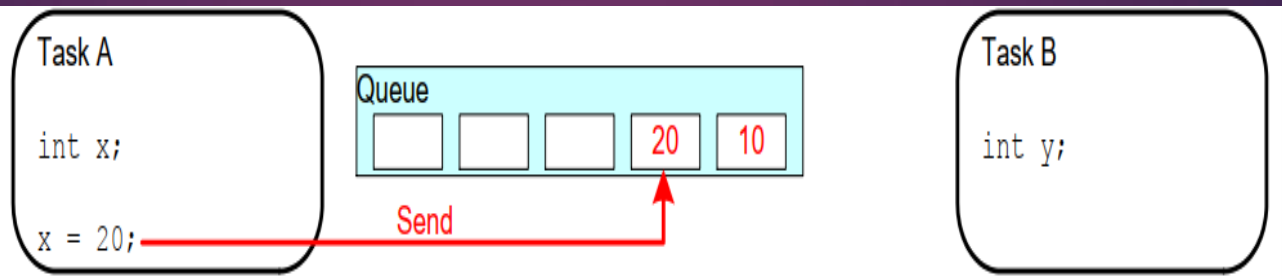


A queue is created to allow Task A and Task B to communicate. The queue can hold a maximum of 5 integers. When the queue is created it does not contain any values so is empty.

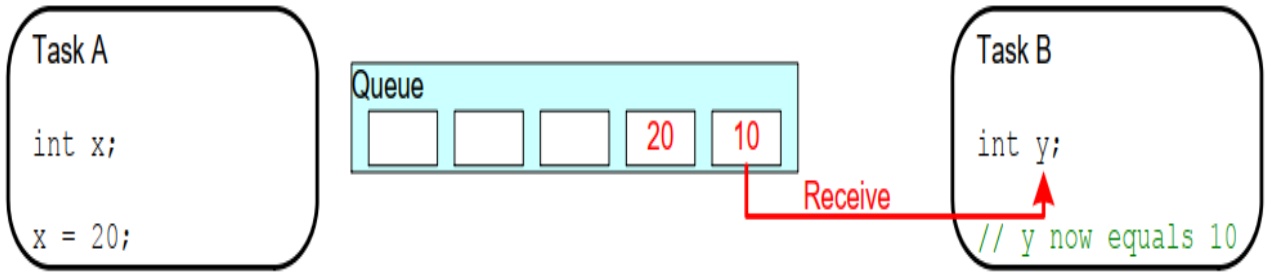


Task A writes (sends) the value of a local variable to the back of the queue. As the queue was previously empty the value written is now the only item in the queue, and is therefore both the value at the back of the queue and the value at the front of the queue.

Queue và xử lý ngắt



Task A changes the value of its local variable before writing it to the queue again. The queue now contains copies of both values written to the queue. The first value written remains at the front of the queue, the new value is inserted at the end of the queue. The queue has three empty spaces remaining.



Task B reads (receives) from the queue into a different variable. The value received by Task B is the value from the head of the queue, which is the first value Task A wrote to the queue (10 in this illustration).

Queue và xử lý ngắt: Một số tình huống

- Ví dụ 1:
  - API **xQueueSendToBack**() có khả năng đưa task gọi nó vào trạng thái blocked nếu queue hiện đang đầy, nhưng nếu API này được gọi từ bên trong ISR của một interrupt thì sao?
- Ví dụ 2:
  - Chương trình có một **Idle Task** (độ ưu tiên 0) và task **Receiver** (độ ưu tiên 2 và trong trạng thái blocked vì phải đợi dữ liệu từ Queue).
  - Mỗi khi có UART RX interrupt của MCU, **ISR** sẽ tạm thời gián đoạn **Idle Task**, API **xQueueSendToBack**() được gọi từ bên trong **ISR** để gửi dữ liệu vừa nhận được vào Queue
  - Mà task **Receiver** cũng đang đợi để đọc dữ liệu, lúc này task Receiver sẽ được chuyển từ trạng thái Blocked vào Ready.
  - Tuy nhiên, vì không có cách nào để Task có thể chiếm quyền thực thi của ISR, nên task Receiver chỉ có thể chờ đợi.
  - Sau khi ISR kết thúc, **Idle Task** (vẫn trong trạng thái Running) tiếp tục được thực thi, lúc này task Receiver mới có thể chiếm quyền thực thi từ Idle Task, đẩy Idle Task vào trạng thái blocked.
- **Chú ý**: Một task đang chạy và bị gián đoạn bởi ISR vẫn sẽ tiếp tục ở trạng thái Running (vì Scheduler cũng bị gián đoạn bởi ISR), nên nó sẽ tiếp tục được thực thi ngay khi ISR kết thúc.

Queue và xử lý ngắt: Một số tình huống

- Ví dụ 3:
  - Trong khi API **xQueueSendToBack**() được gọi từ bên trong **ISR** đang trong quá trình gửi dữ liệu vào Queue thì có một interrupt khác xảy ra (interrupt này có độ ưu tiên cao hơn cả **configMAX_SYSCALL_INTERRUPT_PRIORITY**), ISR của interrupt mới này cũng gửi dữ liệu vào Queue mà **xQueueSendToBack**() đang làm việc trên đó và khiến Queue bị đầy
  - Vậy thì điều gì sẽ xảy ra khi **xQueueSendToBack**() được tiếp tục làm việc?

Queue và xử lý ngắt: Một số tình huống

- Để giải quyết 3 vấn đề trên, FreeRTOS cung cấp bộ **interrupt-safe API** dùng cho ISR
  - Ví dụ như **xQueueSendToBackFromISR**().
  - Các API này có tham số **xHigherPriorityTaskWoken** dùng cho mục đích chuyển đổi ngữ cảnh, tức là thay đổi Running Task. Tham số này phải được khởi tạo là **pdFALSE** và sẽ được set thành **pdTRUE** từ bên trong API (không phải bởi lập trình viên) nếu chuyển đổi ngữ cảnh là cần thiết.
  - Để thực hiện chuyển đổi, sử dụng **portYIELD_FROM_ISR**(xHigherPriorityTaskWoken).
- Ví dụ: ISR và chuyển đổi ngữ cảnh

```
void UARTIntHandler(void) {
    // put Interrupt Handle Code here
    char *string = "UART_ISR";
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    xQueueSendToBackFromISR(semaphore, string, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

Queue và xử lý ngắt: Các hàm API hay dùng

- **Chú ý quan trọng**: Interrupt-safe API
  - Các API có tác động lên tài nguyên (ví dụ như xQueueSendToBack()) đều có sẵn Critical Section bên trong, đảm bảo rằng chỉ có những interrupt nào có độ ưu tiên cao hơn **configMAX_SYSCAL_INTERRUPT_PRIORITY** mới có thể gián đoạn quá trình hoạt động của nó.
  - Các interrupt-safe API còn đảm bảo rằng chúng không thể được sử dụng bởi những interrupt có độ ưu tiên cao hơn **configMAX_SYSCAL_INTERRUPT_PRIORITY**. Do đó Critical Section sẽ được bảo đảm an toàn nếu interrupt-safe API được sử dụng trong các ISR.
- Quay trở lại ví dụ 3:
  - Khi API **xQueueSendToBackFromISR**() được gọi từ bên trong **ISR** để gửi dữ liệu vừa nhận được vào Queue mà task **Receiver** cũng đang đợi để đọc dữ liệu, task **Receiver** sẽ được chuyển từ trạng thái Blocked vào Ready.
  - Vì task Receiver có độ ưu tiên cao hơn Idle Task (hiện đang là Running Task) nên ngữ cảnh được chuyển đổi, task Receiver sẽ trở thành Running Task và Idle Task sẽ bị blocked. Sau khi ISR kết thúc, task Receiver (đã trở thành Running Task) sẽ được thực thi.

Queue và xử lý ngắt: Các hàm API hay dùng

- xQueueSendToFrontFromISR(), xQueueSendToBackFromISR() và xQueueReceiveFromISR() là các phiên bản tương ứng của xQueueSendToFront(), xQueueSendToBack() và xQueueReceive() sử dụng an toàn trong chương trình phục vụ ngắt.
- xQueueSendFromISR() tương đương và giống như xQueueSendToBackFromISR().

FreeRTOS: Deferred Interrupt Processing

- Một nguyên tắc khi sử dụng Interrupt là ISR phải đơn giản nhất có thể, đảm bảo chương trình chính bị gián đoạn ít nhất có thể.
- Trong FreeRTOS, khái niệm “Deferred Interrupt Processing” được đưa ra cũng với mục đích này. ISR có nhiệm vụ xóa cờ ngắt và bàn giao một phần công việc cho một task khác có độ ưu tiên cao hơn task đang bị gián đoạn (bằng cách chuyển đổi ngữ cảnh).
- Deferred Interrupt Processing nên được sử dụng ở các trường hợp sau:
  - khối lượng công việc mà ISR cần xử lý quá lớn
  - khi cần cấp phát bộ nhớ
  - khi thời gian mà ISR có thể kết thúc công việc là không rõ ràng
- Để bàn giao công việc cho task, ISR cần gọi API để unblock task đó và thực hiện chuyển đổi ngữ cảnh. Để unblock một task, có thể sử dụng tín hiệu đồng bộ từ Queue, Binary Semaphore, Counting Semaphore, Mutex, Event Group và Task Notification.

Interrupt Management

- Giới thiệu về ngắt
- Quản lý ngắt dùng Queue
- **Quản lý ngắt dùng Semaphore**

Couting Semaphore

- Để sử dụng Counting Semaphore, cần set **configUSE_COUNTING_SEMAPHORE** bằng 1.
- **Semaphore bản chất cũng là một Queue**.
- Khi có interrupt, ISR sẽ đưa 1 token vào **Counting Semaphore**, Semaphore Counter sẽ tăng 1.
  - Nếu Deferred Task lấy token từ Counting Semaphore, Semaphore Counter sẽ giảm 1.
  - Giá trị của **Semaphore Counter** chính là số lượng interrupt event chưa được xử lý.
- Nếu Semaphore Counter chạm tới giá trị tối đa, nó sẽ không thể nhận thêm token.

Couting Semaphore

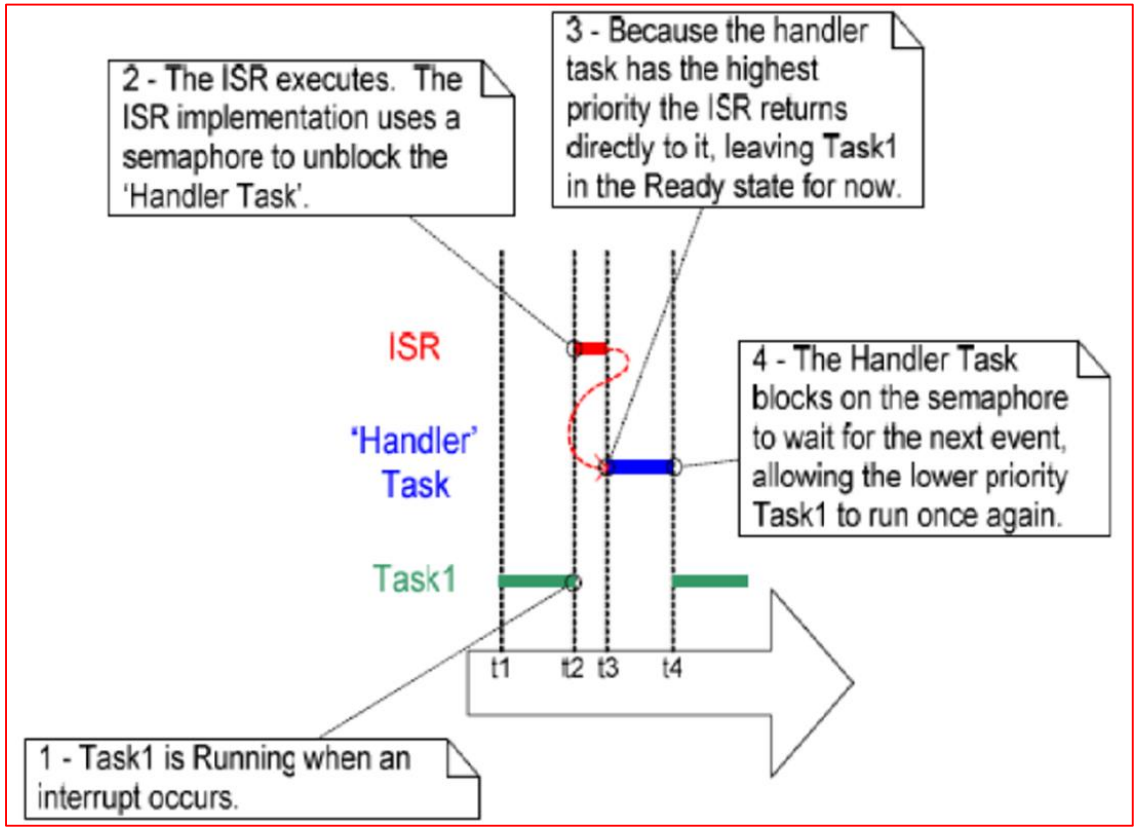
- Giá trị tối đa và giá trị ban đầu của Semaphore Counter được quy định trong API **xSemaphoreCreateCounting**():

```
SemaphoreHandle_t  xSemaphoreCreateCounting(UBaseType_t max_count,
                                            UBaseType_t init_count);
```

- Để đưa và lấy token từ Counting Semaphore, sử dụng API **xSemaphoreGiveFromISR**() và **xSemaphoreTake**().

Binary Semaphore

- **Binary Semaphore** có thể được xem là một **Counting Semaphore** với giá trị tối đa của counter bằng 1.
- ISR sử dụng API **xSemaphoreGiveFromISR**() để đưa token vào queue khi có công việc cần giao cho Deferred Task. Deferred Task sử dụng API **xSemaphoreTake**() để lấy token ra khỏi queue và được unblocked
- Hình vẽ:
  - 1. Tác vụ 1 đang chạy khi một ngắt xảy ra
  - 2. Chương trình con phục vụ ngắt ISR thực thi. Sự thực thi ISR sử dụng một semaphore để mở khóa (Unlock) tác vụ "Handler".
  - 3. Bởi vì tác vụ "Handler" có mức ưu tiên cao nhất nên ISR trả lại trực tiếp cho tác vụ "Handler", Task1 ở trạng thái Ready
  - 4. Tác vụ "Handler" khóa trên semaphore để đợi sự kiện tiếp theo, cho phép tác vụ có mức ưu tiên thấp hơn – Tác vụ 1 – chạy lại.



Binary Semaphore

- Một semaphore nhị phân có thể được sử dụng để mở khóa một tác vụ mỗi khi có ngắt xác định xảy ra, đồng bộ hóa tác vụ với ngắt
- Điều này cho phép hầu hết các xử lý sự kiện ngắt được thực hiện trong tác vụ được đồng bộ hóa, chỉ với một phần rất ít còn lại trực tiếp trong ISR. Xử lý ngắt làm ngưng tác vụ đang xử lý.

Binary Semaphore

- Để tạo một Binary Semaphore, gọi API **xSemaphoreCreateBinary**():

```
SemaphoreHandle_t    xSemaphoreCreateBinary(void);
```

- Để đưa token vào Semaphore, ISR cần gọi API **xSemaphoreGiveFromISR**():

```
BaseType_t   xSemaphoreGiveFromISR(SemaphoreHandle_t semaphore,
                                   BaseType_t  *pxHigherPriorityTaskWoken);
```

  - **pxHigherPriorityTaskWoken:** Biến sẽ quyết định xem có xảy ra contextswitch hay không

- Để lấy token từ Semaphore, Deferred Task cần gọi API **xSemaphoreTake**():

```
BaseType_t  xSemaphoreTake(SemaphoreHandle_t semaphore, TickType_t ticks_to_wait);
```

- Để lấy Token từ Semaphore, ISR cần gọi API **xSemaphoreTakeFromISR**

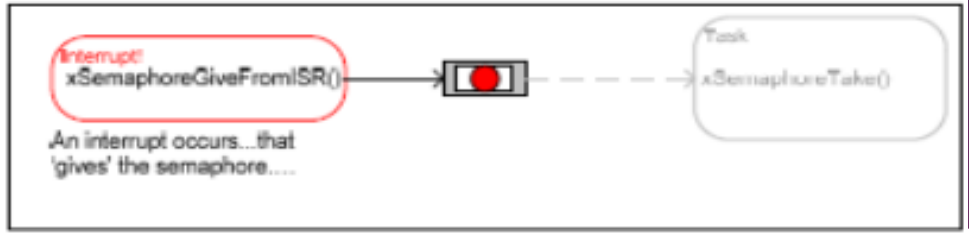
```
xSemaphoreTakeFromISR( SemaphoreHandle_t xSemaphore,
                       signed BaseType_t  *pxHigherPriorityTaskWoken);
```

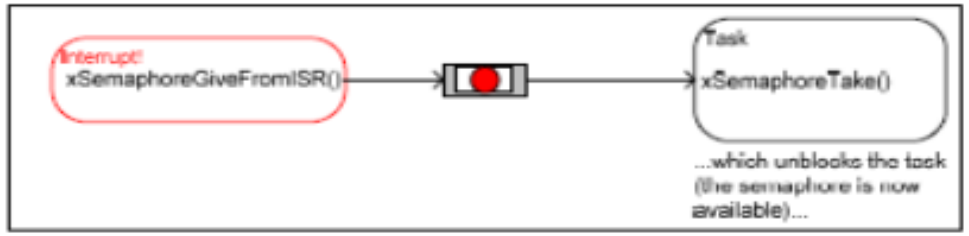
Binary Semaphore

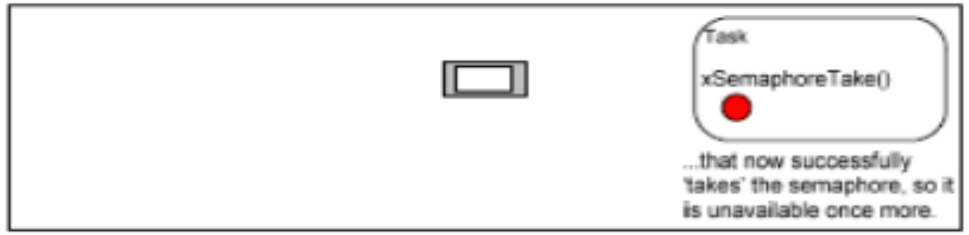
- **Chú ý:**
  - Vì Binary Semaphore chỉ có thể chứa 1 token, nên nếu có interrupt xảy ra trong khi Deferred Task đang xử lý công việc và Binary Semaphore đang đầy thì những interrupt event đó sẽ bị mất. Để khắc phục vấn đề này, Deferred Task phải được thiết kế để có khả năng xử lý tất cả sự kiện xảy ra giữa 2 lần gọi **xSemaphoreTake**().
- Ví dụ:
  - Khi được ISR của UART RX interrupt bàn giao công việc, Deferred Task phải đọc cho đến khi không còn dữ liệu trong UART buffer, để tránh bỏ sót dữ liệu vì có thể trong khi Deferred Task đang xử lý dữ liệu thì đã có thêm nhiều ký tự được gửi đến, UART RX interrupt đã xảy ra nhưng token không được ghi thêm vào Binary Semaphore.

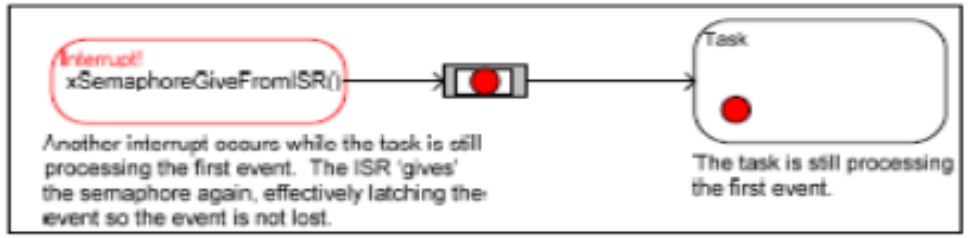
Binary Semaphore

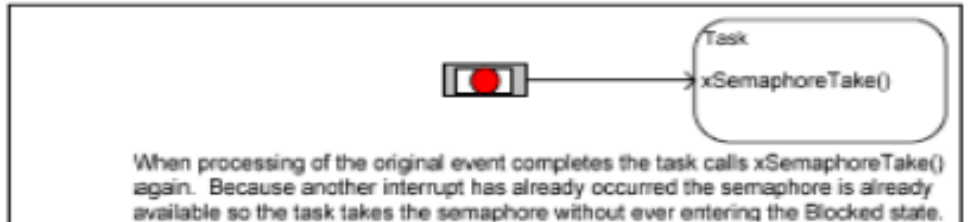




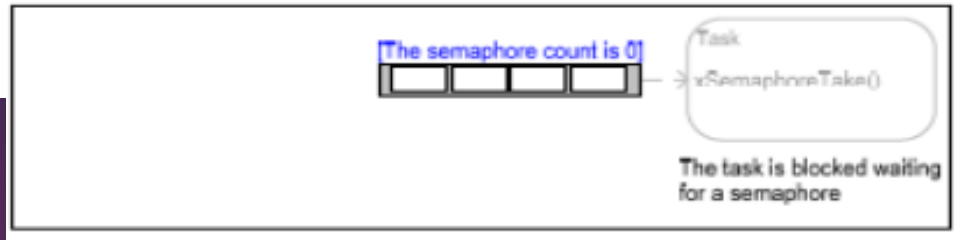


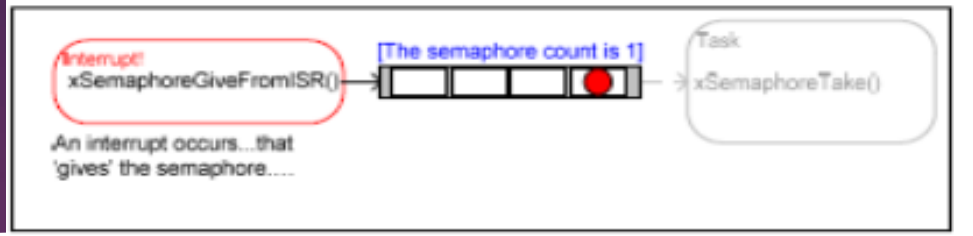


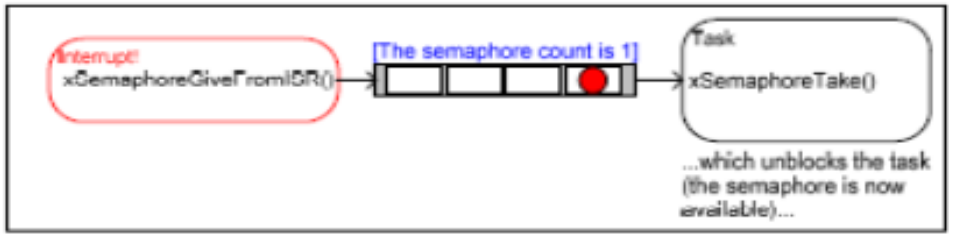


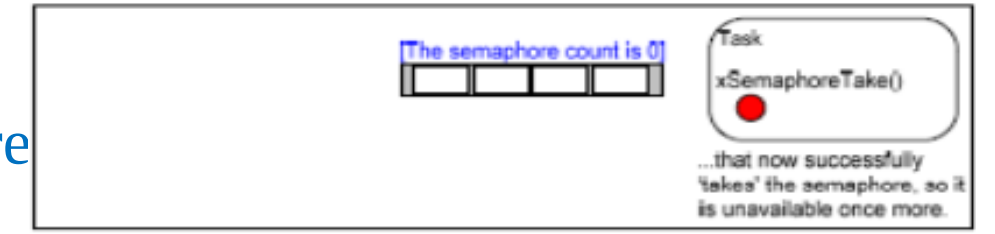


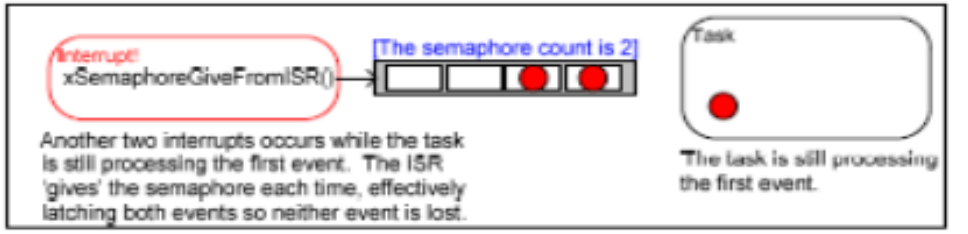
Counting Semaphore

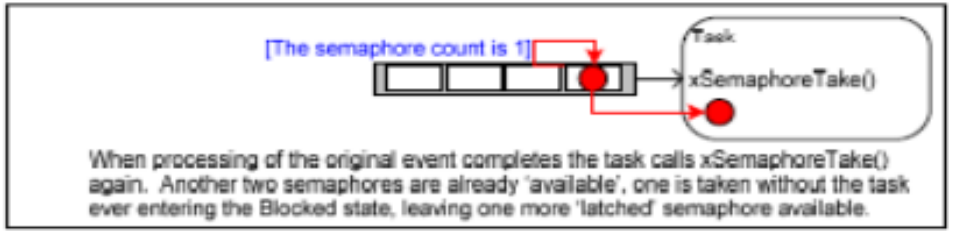




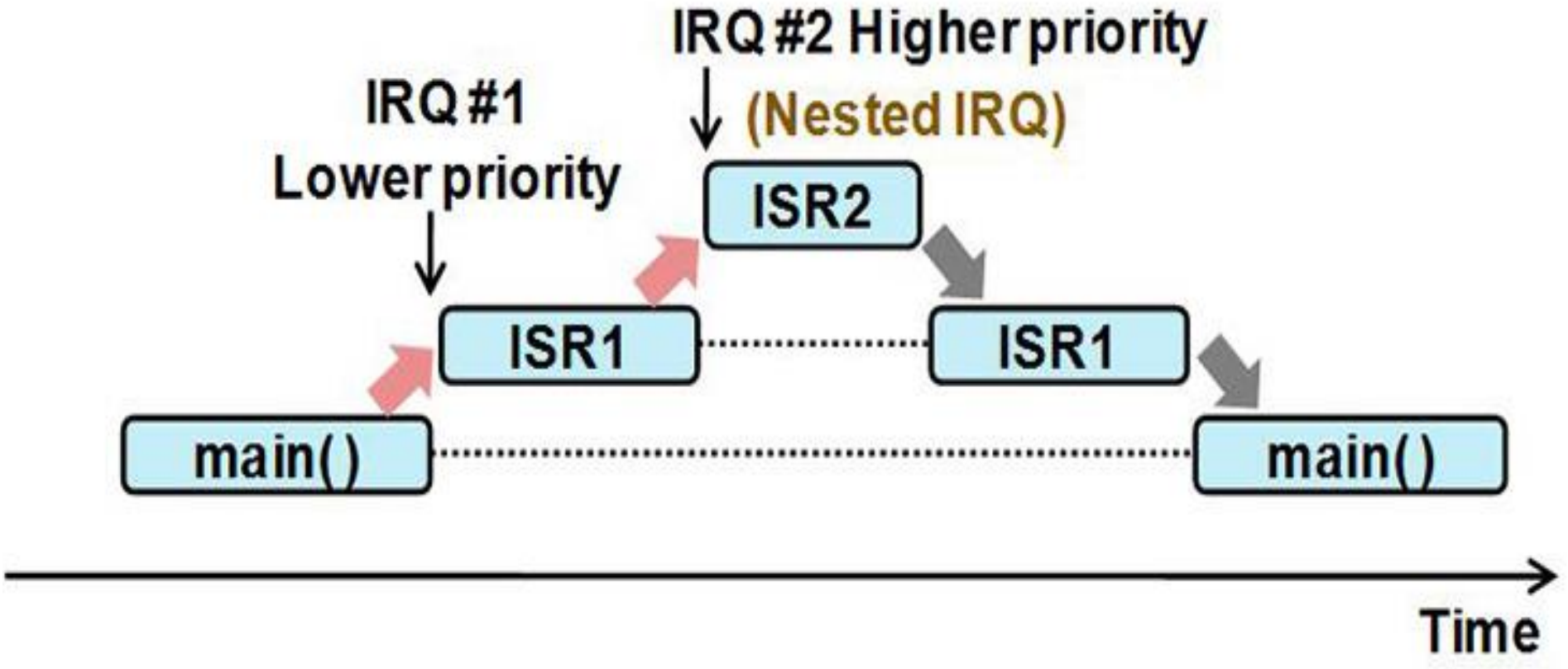








Mô hình lồng ngắt



Mô hình lồng ngắt

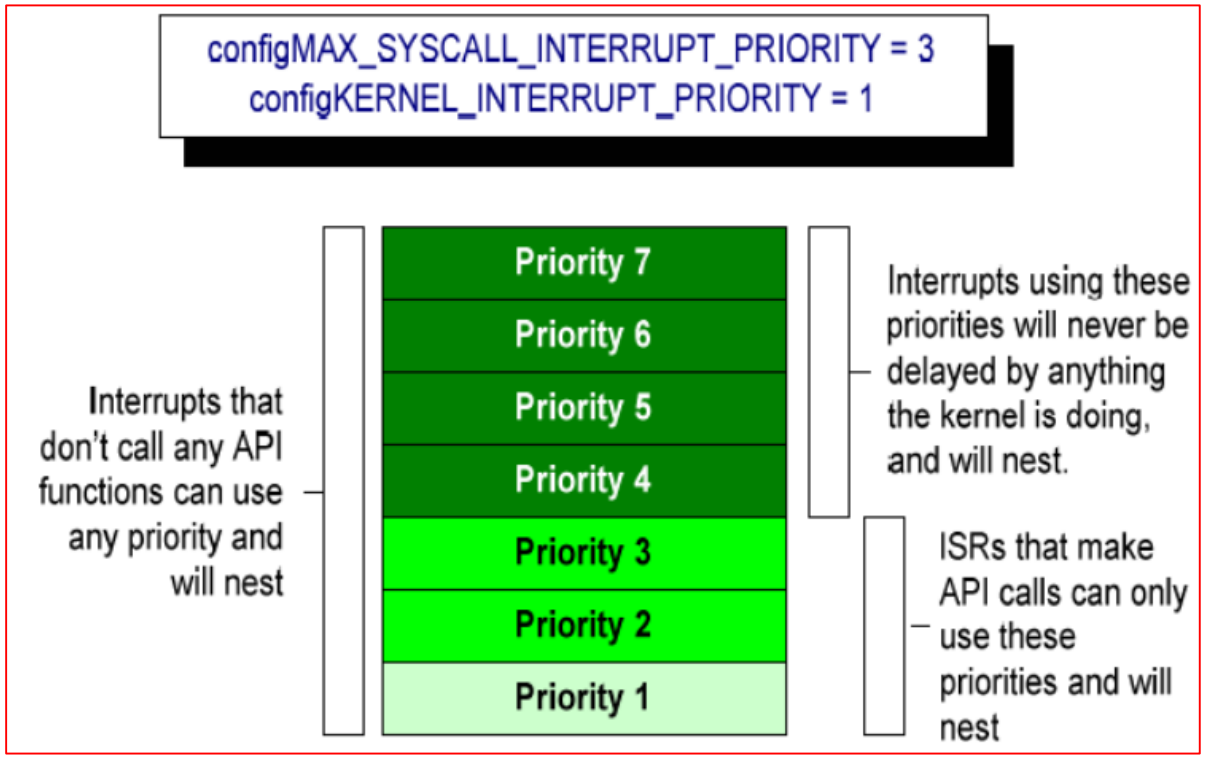
- Một mô hình lồng ngắt hoàn chỉnh được tạo ra bởi việc thiết lập **configMAX_SYSCALL_INTERRUPT_PRIORITY** ở mức ưu tiên cao hơn **configKERNEL_INTERRUPT_PRIORITY**

Bảng 3.6: Các hằng số kiểm soát việc lồng các ngắt.

Hằng số	Mô tả
`configKERNEL_INTERRUPT_PRIORITY`	Thiết lập mức ưu tiên ngắt sử dụng bởi ngắt đánh dấu. Nếu cổng không dùng hằng `configMAX_SYSCALL_INTERRUPT_PRIORITY` thì bất kì ngắt nào sử dụng hàm API FreeRTOS an toàn ngắt cũng phải thực thi ở mức ưu tiên này.
`configMAX_SYSCALL_INTERRUPT_PRIORITY`	Thiết lập mức ưu tiên ngắt cao nhất từ các hàm API FreeRTOS an toàn ngắt được gọi.

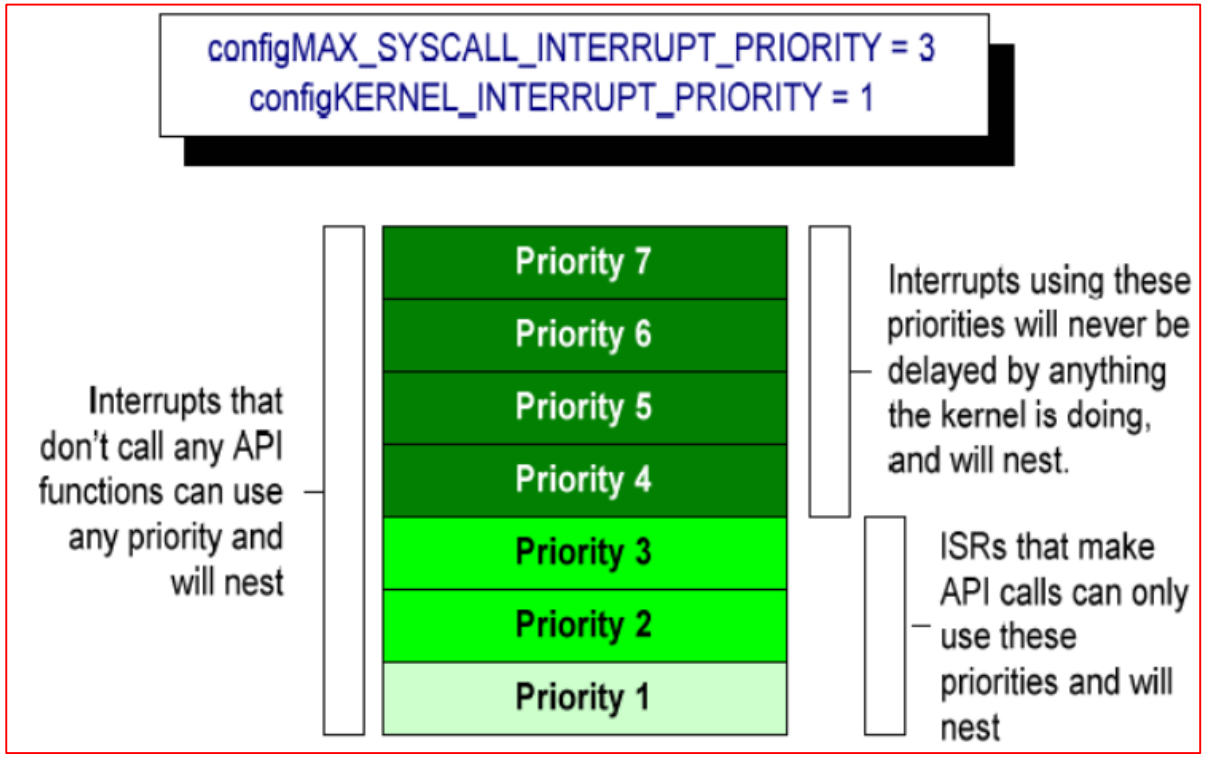
Mô hình lồng ngắt

- Sự nhầm lẫn phát sinh giữa các mức ưu tiên tác vụ và các mức ưu tiên ngắt là rất phổ biến
- Hình bên cho thấy các mức ưu tiên ngắt:
  - Được định nghĩa bởi kiến trúc vi xử lí. Đây là những mức ưu tiên được kiểm soát phần cứng mà các ISR thực thi liên quan đến nhau.
  - Các tác vụ không chạy trong các ISR nên mức ưu tiên mềm được gán cho một tác vụ không liên quan đến mức ưu tiên cứng được gán cho một nguồn ngắt



Mô hình lồng ngắt

- Tham chiếu hình:
  - Các ngắt sử dụng các mức ưu tiên từ 1 đến 3 sẽ bị ngăn cản thực hiện trong khi kernel hoặc ứng dụng nằm trong một section quan trọng (CS), nhưng chúng có thể sử dụng các hàm API an toàn ngắt FreeRTOS
  - Các ngắt sử dụng mức ưu tiên 4 hoặc cao hơn không bị ảnh hưởng bởi các CS, do đó không có kernel nào ngăn chặn các ngắt này thực thi ngay lập tức - trong giới hạn của chính bộ vi điều khiển. Thông thường, function đòi hỏi độ chính xác thời gian rất nghiêm ngặt (ví dụ như điều khiển động cơ) sẽ sử dụng mức ưu tiên ở trên configMAX_SYSCALL_INTERRUPT_PRIORITY để đảm bảo trình lập lịch không giới thiệu jitter vào thời gian phản hồi các ngắt.
  - Các ngắt không thực hiện bất kỳ lời gọi hàm API FreeRTOS nào đều rảnh rỗi để sử dụng bất kỳ mức độ ưu tiên nào



Bài tập thực hành: TH7 – Interrupt Management

- TH7.1: Arduino Interrupt example

https://microcontrollerslab.com/use-arduino-interrupts-examples/

- TH7.2: Using Binary Semaphore: Task – Interrupt Synchronization

https://microcontrollerslab.com/freertos-binary-semaphore-tasks-interrupt-synchronization-u-arduino/ (Ví dụ 2)

Q & A

Câu hỏi ôn tập:

1. Liệt kê bảng vectơ ngắt trên Arduino Uno với các mức ưu tiên tương ứng
2. So sánh semaphore nhị phân và semaphore đếm. Mô tả ngắn gọn các trường hợp sử dụng semaphore đếm trong hệ thống sử dụng FreeRTOS
3. Hãy nêu khái niệm mutex. Hãy so sánh semaphore nhị phân và mutex.

Bài tập thực hành: TH7 – Nâng cao

- TH7.3: Handle Interrupt

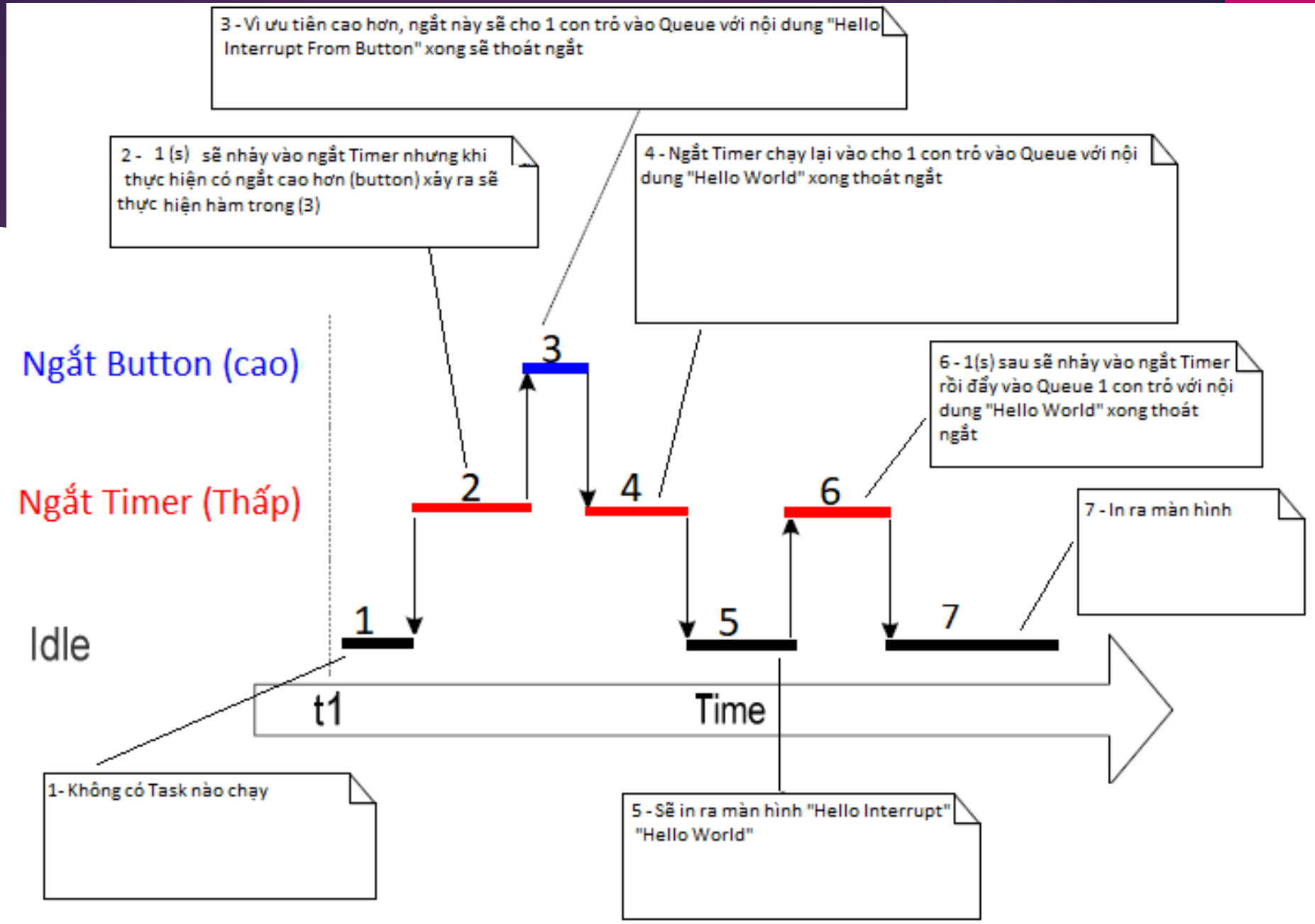
https://exploreembedded.com/wiki/Resuming_Task_From_ISR

- TH7.4: Interrupt management using Queue

https://microcontrollerslab.com/freertos-interrupt-management-examples-with-arduino/

- TH7.5: Viết chương trình sử dụng 2 ngắt lồng nhau thực hiện như hình sau:
  - Sau 1s sẽ in “Hello World” lên màn hình.
  - Ấn nút sẽ in “Hello Interrupt From Button”

TH7.5:


3 - Vì ưu tiên cao hơn, ngắt này sẽ cho 1 con trỏ vào Queue với nội dung "Hello Interrupt From Button" xong sẽ thoát ngắt
2 - 1 (s) sẽ nhảy vào ngắt Timer nhưng khi thực hiện có ngắt cao hơn (button) xảy ra sẽ thực hiện hàm trong (3)
4 - Ngắt Timer chạy lại vào cho 1 con trỏ vào Queue với nội dung "Hello World" xong thoát ngắt
Ngắt Button (cao)
Ngắt Timer (Thấp)
Idle
6 - 1(s) sau sẽ nhảy vào ngắt Timer rồi đẩy vào Queue 1 con trỏ với nội dung "Hello World" xong thoát ngắt
7 - In ra màn hình
1
2
3
4
5
6
7
t1
Time
1- Không có Task nào chạy
5 - Sẽ in ra màn hình "Hello Interrupt" "Hello World"